F2: Engine Logic & Technical Whitepaper

By Basel Shokha

Problem Definition

Input:
• A weighted directed graph $G = (V, E)$ with a non-negative weight function $w: E \to \mathbb{R}^+$, representing the regional map and road distances.
• A set of autonomous fleet units $U = \{u_1, u_2, ..., u_n\}$ located at specific nodes in V.
• A set of target tasks or destinations $T = \{t_1, t_2, ..., t_m\}$ located at specific nodes in V. The dispatch system prevents spawning excess units, guaranteeing $|U| \geq |T|$.

Output:
An optimal bipartite matching (assignment set) $M \subseteq U \times T$ such that the maximum possible number of tasks are fulfilled, meaning $|M| = |T|$, and the global sum of assignment path weights in G is strictly minimized.

Algorithm Description

Definition (Flow Network). The multi-agent fleet assignment is mathematically modeled as a Min-Cost Max-Flow network $N = (V_N, E_N)$ with a global source s and a global sink t. Let $w(u_i, t_j)$ denote the total weight of the lightest path in G from fleet unit u_i to task t_j.

1. Define the vertices as $V_N = \{s, t\} \cup U \cup T$. Construct the directed edges E_N as follows:
• Edges from s to each $u_i \in U$ with capacity 1 and cost 0.
• Edges from each $u_i \in U$ to each $t_j \in T$ with capacity 1 and cost $w(u_i, t_j)$.
• Edges from each $t_j \in T$ to t with capacity 1 and cost 0.

2. Execute the Min-Cost Max-Flow algorithm to push maximum integral flow from s to t while actively maintaining node potentials (used for Dijkstra) to minimize the total cost of the routing flow.

3. For every intermediate edge (u_i, t_j) that carries a positive flow of exactly 1, extract its physically **induced path** in G and add the corresponding pair to the assignment set M.

4. Return M.

Proof of Correctness

Lemma 1: The maximum flow guarantees a maximal valid bipartite matching.

Proof: By defining capacities of all edges connected to the global source s and sink t as exactly 1, the integral flow theorem ensures the maximum flow consists entirely of discrete paths carrying exactly 1 unit of flow. By the strict mathematical laws of flow conservation, each unit u_i can emit at most 1 unit of flow, and each task t_j can receive at most 1 unit. Thus, the saturated paths directly correspond to a set of independent edges between U and T, forming a valid bipartite matching. A max-flow execution formally guarantees that exactly $|T|$ assignments are made (since $|U| \geq |T|$). ■

Claim: The assignment set *M* returned by the algorithm is mathematically optimal with respect to minimizing travel distance in *G*.

Proof: Let M' be any alternative maximal assignment with an overall weight W'. Initially, M' trivially can be matched to a flow assignment on our flow network N. By its correctness, executing the Min-Cost Max-Flow algorithm on N guarantees a result assignment M that has a weight less than or equal to the weight of M'. By the Min-Cost Max-Flow theorem, an integral flow achieves absolute minimum weight if and only if there are no negative-weight augmenting cycles in the residual network N_f. The flow algorithm pushes flow strictly along the lightest augmenting paths, utilizing Johnson's node potentials to handle dynamic edge weights. Therefore, upon reaching maximum flow, the residual graph is guaranteed to be cycle-free with respect to negative weights. This formally implies the total weight of the final flow $W \leq W'$. Because W maps perfectly to the sum of weights for M, the algorithm guarantees an optimal fleet distribution. ■

Time Complexity Analysis

Step 1 : Deriving the cost matrix requires finding the lightest paths in G from each fleet unit. Running Dijkstra's algorithm from every $u_i \in U$ using a standard minimum heap takes $O(|U| \cdot |E| \log |V|)$.
Constructing the dense bipartite network N takes bounded time $O(|U| \cdot |T|)$.

Step 2 : Because the system prevents spawning robots less than needed, we are guaranteed $|U| \geq |T|$. Therefore, the maximum flow is exactly $|f^*|= |T|$, requiring $|T|$ flow augmentations. Each augmentation requires running Dijkstra's algorithm with potentials over V_N nodes and E_N edges. Since $|V_N| = |U| + |T| + 2$ and $|E_N| = |U| + |T| + |U||T|$, each run using a minimum heap takes $O(|U||T| \log (|U|+|T|))$. The total optimization step costs $O(|U||T|^2 \log (|U|+|T|))$.

Total Time Complexity: (steps 3 and 4 are negligible)
Combining the map traversal and flow augmentation phases, the exact complexity evaluates to:
$\boldsymbol{O(|U| \cdot |E| \log |V| \ + \ |U||T|^2 \log (|U|+|T|))}$

In a real-world dispatch environment, the regional road network is massive compared to the fleet size ($|E|, |V| \gg |U|, |T|$). Under these conditions, the routing term is completely negligible and can be safely omitted. The overall engine performance is dominated entirely by the initial graph traversal, yielding an effective runtime of:
$\boldsymbol{O(|U| \cdot |E| \log |V|)}$